HDOC-63

Missing Number in an Integer Array Using Pointers

Problem Statement

Write a C program that reads the size and elements of a variable-length integer array nums[]. The array contains distinct integers from 0 to size, with exactly one number missing. Pass the array to a function named missing_number() using its address. Use pointers to pass the array between the calling function and missing_number(). The function must find and return the missing integer, which is then displayed as an integer.

Requirements

- Find the missing integer from the range 0 to size.
- Display the missing number.
- Assume all elements in the array are distinct.
- Use the fact that the array contains exactly size numbers from the range 0 to size, with one number missing.
- Use pointers to pass the array to the function.

Algorithm / Logic

1. Start the program.

2. Read size, the number of elements present in the array.

3. Declare the variable-length array nums[size].

4. Read size distinct integers, each belonging to the range 0 to size.

5. Call missing_number(nums, size). Since an array name represents the address of its first element, the function receives the array through an integer pointer.

6. Inside missing_number(), calculate the expected sum of all integers from 0 to size using size * (size + 1) / 2.

7. Traverse the array using pointer arithmetic and calculate the sum of the entered elements.

8. Subtract the actual sum from the expected sum. The difference is the missing number.

9. Return the missing number to main(), display it, and stop.

C Program

```c
#include <stdio.h>

int missing_number(int *nums, int size)
{
    int expected_sum = size * (size + 1) / 2;
    int actual_sum = 0;

    for (int i = 0; i < size; i++)
    {
        actual_sum += *(nums + i);
    }

    return expected_sum - actual_sum;
}

int main()
{
    int size;

    printf("Enter the size of the array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Size must be a positive integer.\n");
        return 1;
    }

    int nums[size];

    printf("Enter %d distinct elements from 0 to %d: ", size, size);
    for (int i = 0; i < size; i++)
    {
        scanf("%d", nums + i);
    }

    int missing = missing_number(nums, size);
    printf("Missing number = %d\n", missing);

    return 0;
}
```

Explanation

If the complete range is 0 to size, its sum is size(size + 1)/2. Because exactly one value is absent, subtracting the sum of the entered array elements from this expected sum gives the missing value. The parameter int *nums stores the address of the first array element. The expression

*(nums + i) accesses the i-th element through pointer arithmetic, satisfying the pointer requirement.

Time Complexity: O(n), because the array is traversed once.
Space Complexity: O(1) auxiliary space, excluding the input variable-length array.

Test Cases

Test Case	Size	Array Elements	Expected Missing Number
1	5	0 1 3 4 5	2
2	4	0 1 2 3	4
3	6	1 2 3 4 5 6	0
4	7	0 1 2 3 5 6 7	4

Sample Output

```
Enter the size of the array: 5
Enter 5 distinct elements from 0 to 5: 0 1 3 4 5
Missing number = 2
```

Result

The program successfully reads a variable-length integer array, passes its address to missing_number() using a pointer, finds the single missing integer from the range 0 to size, returns it to the calling function, and displays the result.